

Modular Coordinate Transformations in Integer Factorization

From Close-Modulus Filtering to the Boundary Between Constraint and Factor Generation

IamLupo

30 August 2026

Abstract

This paper documents a sequence of mathematical experiments investigating whether several closely spaced prime moduli can provide a new route to factoring a composite integer. The central idea was to exploit the simultaneous congruences

$$pq \equiv n \pmod{r_i}$$

for several nearby primes r_i , and to combine them with the hyperbolic relation $pq = n$. A series of coordinate changes was explored: residue–quotient coordinates, quotient-product coordinates, hyperbola strips, residue lifts, quotient-cell defects, batch divisor isolation, Chinese-remainder reconstruction, and symbolic constraint solving.

The experiments consistently revealed a common mechanism. Modular equations became increasingly strong as filters and became particularly restrictive when combined with geometric bounds from $q = n/p$. However, the transformations did not demonstrate a mechanism that generates the location of an unknown factor without first introducing, explicitly or implicitly, the corresponding search space. Several apparent collapses were shown to be exact reformulations of divisibility or of the original factorization equation.

The principal conclusion is therefore not that modular information is useless, but that the tested close-modulus constructions primarily provide *relational information*: once a candidate factor or coordinate is supplied, the other coordinates can often be constrained or reconstructed very efficiently. The unresolved step is the transition from relational filtering to independent candidate generation. This distinction gives a precise boundary for future work and clarifies why an additional mathematical engine is required if the goal is to obtain a fundamentally new factoring method.

Contents

1	Introduction	2
2	Initial Hypothesis	2
3	Residue–Quotient Coordinates	3
4	Close-Modulus Linearization	4
5	The Mixed-Modulus Representation	4
6	Hyperbola Geometry	5
7	Residue Lifting	5

8	The Quotient-Cell Defect	6
9	Batch Divisor Isolation	7
10	Chinese-Remainder Reconstruction	7
11	The Exact t -Parameter	8
12	Direct Constraint Solving	8
13	The Central Mechanism: Filtering Versus Generation	9
14	Why the Close Moduli Did Not Break the Symmetry	9
15	Information Content and Coordinate Changes	9
16	What the Experiments Actually Established	10
17	What Is Missing?	10
18	Relation to Broader Factoring Methods	11
19	Future Research Direction	11
20	Conclusion	12

1 Introduction

Integer factorization begins with a deceptively simple equation. Given a composite integer

$$n = pq,$$

where p and q are unknown factors, the objective is to recover p and q from n alone.

The research described here started from a different perspective. Instead of approaching the factorization problem through a conventional integer search, it considered several auxiliary prime moduli

$$r_1, r_2, r_3, \dots$$

chosen to be relatively close to one another. The motivation was that nearby moduli have related arithmetic structure. If the same unknown factors are viewed modulo several such primes, perhaps the small differences between the moduli could force the factor coordinates into a substantially smaller region than an ordinary factor search would require.

This led to a progression of increasingly elaborate representations. The original factor pair (p, q) was replaced by residue and quotient coordinates. The product equation was then combined with a hyperbola geometry, transformed into quotient-cell conditions, rewritten using defects and remainders, and finally represented as Chinese-remainder or solver constraints.

The research question was not merely whether these constructions can identify the correct factor once a candidate is supplied. That is comparatively easy. The deeper question is:

Can the modular structure itself *generate* a small set of possible factor locations, rather than merely *filter* a candidate space that was generated by some other process?

The experiments did not establish such a generation mechanism. Nevertheless, they exposed several useful mathematical facts about where the information goes when the representation is changed, why some apparent breakthroughs collapse under exact substitution, and what kind of new ingredient would be needed to cross the remaining boundary.

2 Initial Hypothesis

The central hypothesis was that several close prime moduli might contain more useful information jointly than they appear to contain individually.

For any modulus r_i coprime to p , the equation

$$pq = n$$

implies

$$pq \equiv n \pmod{r_i}.$$

Hence

$$q \equiv np^{-1} \pmod{r_i}.$$

At first sight, using many moduli could seem to create multiple independent restrictions on p and q . When the moduli are close,

$$r_2 = r_1 + \Delta_2, \quad r_3 = r_1 + \Delta_3,$$

there are also simple identities connecting the residues of the same integer under the different moduli.

The initial expectation was therefore that the system

$$pq \equiv n \pmod{r_1}, \quad pq \equiv n \pmod{r_2}, \quad pq \equiv n \pmod{r_3}$$

combined with the small differences between the r_i might constrain the unknown factors strongly enough that the factor search itself could be reduced to a small number of candidates.

The experiments were designed to determine whether this expectation was genuine or whether the additional equations merely represented the same unknowns in increasingly compressed coordinates.

3 Residue–Quotient Coordinates

A natural first transformation is

$$p = a + kr_1, \quad q = b + \ell r_1,$$

with

$$0 \leq a, b < r_1.$$

Here a and b are residues and k and ℓ are quotient coordinates.

The product equation becomes

$$n = (a + kr_1)(b + \ell r_1) \tag{1}$$

$$= ab + a\ell r_1 + bkr_1 + k\ell r_1^2. \tag{2}$$

Modulo r_1 this reduces to

$$ab \equiv n \pmod{r_1}.$$

Thus, for invertible a ,

$$b \equiv na^{-1} \pmod{r_1}.$$

This is already an important structural observation. One residue can determine the other residue modulo the chosen modulus. The apparent two-dimensional residue problem is therefore relational rather than fully independent.

If

$$g = n \bmod r_1,$$

then there exists an integer t such that

$$n = g + tr_1.$$

Likewise, because

$$ab \equiv g \pmod{r_1},$$

there exists h such that

$$ab = g + hr_1.$$

Consequently,

$$ab - n = (h - t)r_1.$$

This identity became central to the later coordinate manipulations. It also exposed an early source of error: the residue $g = n \bmod r_1$ is not generally the residue of n modulo a different modulus r_2 . The full quotient information of both n and ab must be retained when changing modulus.

4 Close-Modulus Linearization

Suppose now that

$$p = a + kr_1, \quad q = b + \ell r_1,$$

and define

$$\Delta = r_2 - r_1.$$

Then

$$p \equiv a - k\Delta \pmod{r_2},$$

and similarly

$$q \equiv b - \ell\Delta \pmod{r_2}.$$

Applying the product congruence modulo r_2 gives

$$(a - k\Delta)(b - \ell\Delta) \equiv n \pmod{r_2}.$$

Using

$$ab - n = (h - t)r_1,$$

and expanding yields the exact relation

$$(h - t)r_1 - a\ell\Delta - bk\Delta + k\ell\Delta^2 \equiv 0 \pmod{r_2}.$$

For fixed a, b, k , this is linear in ℓ :

$$\ell(k\Delta^2 - a\Delta) \equiv bk\Delta - (h - t)r_1 \pmod{r_2}.$$

This looked promising because one quotient variable could be solved from another. However, the meaning of the reduction is important: solving for ℓ does not remove the need to consider the possible values of a and k . It changes a two-coordinate search into a one-coordinate family plus reconstruction.

In other words,

$$(a, k, \ell)$$

contains the same underlying factor location as

$$(p, q).$$

The equation has become easier to evaluate for fixed coordinates, but the factor location has not disappeared.

5 The Mixed-Modulus Representation

A different representation uses different modulus scales for the two factors:

$$p = a + kr_1, \quad q = b + \ell r_2,$$

with

$$0 \leq a < r_1, \quad 0 \leq b < r_2.$$

Then

$$n = (a + kr_1)(b + \ell r_2) \tag{3}$$

$$= ab + a\ell r_2 + bkr_1 + k\ell r_1 r_2. \tag{4}$$

If

$$r_1 r_2 r_3$$

is close to n , the dominant product term suggests a quotient relation of the form

$$k\ell \approx r_3.$$

This motivates a quotient-product coordinate system in which the pair (k, ℓ) lies near a hyperbola. Such a representation can be substantially smaller than the full rectangular quotient lattice for the bounded examples that motivated the research.

However, the crucial issue remains: a smaller representation is not automatically an algorithmic breakthrough. The quotient coordinates are simply another encoding of the unknown factor pair.

6 Hyperbola Geometry

The exact factor relation

$$pq = n$$

places the factor pair on the hyperbola

$$q = \frac{n}{p}.$$

For a fixed quotient coordinate k under

$$p = a + kr_1,$$

the factor p belongs to the interval

$$kr_1 \leq p < (k+1)r_1.$$

Therefore the corresponding q satisfies

$$\frac{n}{(k+1)r_1} < q \leq \frac{n}{kr_1}.$$

This means that for each k , only a narrow interval of possible q values needs to be considered. The two-dimensional quotient lattice is therefore replaced by a set of one-dimensional strips following the hyperbola.

This geometric reduction is genuine, but it is also transparent: it comes directly from the equation $pq = n$. The hyperbola does not reveal a hidden factor location; it merely expresses the reciprocal relationship between the factors.

The resulting question becomes more precise:

Can modular information collapse the hyperbola strips before the factor variable itself is enumerated?

7 Residue Lifting

For a fixed candidate factor coordinate

$$p = kr_1 + a,$$

one can use the first modular equation

$$pq \equiv n \pmod{r_1}$$

to determine a compatible residue of q .

When a is invertible modulo r_1 ,

$$q \equiv na^{-1} \pmod{r_1}.$$

This suggests a residue-lifting procedure: instead of scanning every integer q in a hyperbola cell, generate only those values compatible with the first modulus, and then apply additional moduli as filters.

The difficulty observed in the research is that the modular lift can create many residue states even though only a very small number survive later constraints. The strong final collapse is real, but the intermediate state generation can exceed the original factor-prime search.

Thus the modular system is extremely effective at rejecting candidates, yet the rejection occurs after a potentially large candidate family has been generated.

8 The Quotient-Cell Defect

Another attempt was to exploit the exact cell structure directly. Write

$$p = kr_1 + a, \quad q = \ell r_2 + c,$$

and define the cell defect

$$D = n - k\ell r_1 r_2.$$

Then

$$D = kr_1 c + \ell r_2 a + ac. \tag{5}$$

Solving for a gives

$$a = \frac{D - kr_1 c}{\ell r_2 + c}.$$

At first glance, this appears to be a strong divisibility condition producing only a small number of admissible residues a . But because

$$D = n - k\ell r_1 r_2,$$

substitution gives

$$D - kr_1 c = n - kr_1(\ell r_2 + c). \tag{6}$$

Hence

$$a = \frac{n - kr_1 c}{\ell r_2 + c} = \frac{n}{\ell r_2 + c} - kr_1.$$

Therefore exact divisibility is equivalent to

$$q \mid n.$$

This was one of the clearest negative results in the research. A seemingly new arithmetic collapse was simply another form of the original factor condition.

The lesson is general: whenever a newly introduced quotient or defect variable produces a very strong exact division condition, the first check must be whether that condition is merely the original statement $q \mid n$ expressed in transformed coordinates.

9 Batch Divisor Isolation

The hyperbola intervals suggest another optimization. For a fixed interval of possible q values, instead of testing each q separately for divisibility of n , one can aggregate many candidates and use a product or greatest-common-divisor construction to detect whether the interval contains a divisor.

Conceptually, for a set of candidates

$$q_1, q_2, \dots, q_m,$$

one may consider

$$G = \gcd(n, q_1 q_2 \cdots q_m).$$

If $G = 1$, the batch contains no divisor of n . If $G > 1$, the interval can be recursively subdivided.

This produces a substantial reduction in the number of expensive gcd operations. But it does not remove the underlying information-processing requirement if all candidate q_i must first be incorporated into the batch product.

The experiment therefore exposed an important complexity distinction:

$$\boxed{\text{fewer gcd calls} \neq \text{fewer candidate values generated.}}$$

Batching can improve implementation efficiency while leaving the underlying search space unchanged.

10 Chinese-Remainder Reconstruction

For a candidate p coprime to several moduli, each congruence gives

$$q \equiv np^{-1} \pmod{r_i}.$$

If the moduli are pairwise coprime, the Chinese remainder theorem combines these into a single class

$$q \equiv Q_p \pmod{R},$$

where

$$R = r_1 r_2 r_3.$$

The corresponding hyperbola interval can then be intersected with one arithmetic progression.

Equivalently, if the hyperbola interval is

$$q_{\min} \leq q \leq q_{\max},$$

one can ask whether it contains any integer satisfying

$$q \equiv Q_p \pmod{R}.$$

For suitably chosen parameters, the interval can contain zero or one such residue representative. This makes the modular filter extremely selective.

Nevertheless, the construction still depends on the initial p . The CRT class is a function of the candidate supplied to it:

$$p \longmapsto Q_p.$$

The CRT mechanism therefore reflects a candidate rather than originating one.

11 The Exact t -Parameter

Combining the factor equation with a combined modulus R gives

$$pq = n + tR$$

for some integer t .

At first this introduces a new variable that might appear useful. But the hyperbola immediately gives

$$pq \leq n,$$

and, for bounded positive factors,

$$n - pq < p.$$

Therefore if

$$R > p_{\max} - 1,$$

then the only multiple of R compatible with the defect interval is zero, forcing

$$t = 0.$$

Thus

$$pq \equiv n \pmod{R}$$

combined with the hyperbola and the size bound reduces exactly to

$$pq = n.$$

This is mathematically useful because it identifies a limit of the quotient parameter. The modular variable t does not provide an additional hidden coordinate in this regime; the geometry collapses it to a single exact value.

12 Direct Constraint Solving

A further change of viewpoint was to stop explicitly enumerating candidates and instead provide the arithmetic constraints to a symbolic solver.

The basic model is

$$pq = n,$$

possibly together with factor bounds and modular constraints such as

$$p \equiv A_i \pmod{r_i}, \quad q \equiv B_i \pmod{r_i}.$$

When the correct residues are supplied, a constraint solver can recover the corresponding factor pair efficiently in the tested small instances. However, supplying the true residues changes the problem: the location information has already been given to the solver.

The more significant formulation leaves the residues unknown. Then the solver must simultaneously determine

$$p, q, A_i, B_i$$

subject to the factorization and modular relations.

The resulting difficulty demonstrated an important conceptual point. A symbolic solver is not automatically a factorization oracle. Removing an explicit Python loop over candidate p does not prove that the search has disappeared. The solver may simply be performing the same combinatorial search internally.

This leads to a useful distinction:

An explicit enumeration is replaced by a symbolic search only when the constraint system exposes a structure that the solver can exploit more efficiently than generic search.

The experiments did not establish such a scalable advantage for the unknown-residue formulations.

13 The Central Mechanism: Filtering Versus Generation

The sequence of experiments converged on one common mechanism.

Suppose the unknown factor p is supplied. Then modular arithmetic gives a remarkably powerful response:

$$q \equiv np^{-1} \pmod{R}.$$

Geometric information then restricts q to an interval, and several moduli can reduce the compatible set to only a few possibilities.

The direction of information flow is therefore

$$p \longrightarrow q \pmod{R} \longrightarrow \text{small compatible set}.$$

The desired direction for a new factor-generation method would instead be

$$n \longrightarrow \text{small candidate set for } p \longrightarrow q = n/p.$$

The tested modular equations did not provide the second implication.

This is the essential boundary encountered throughout the research.

14 Why the Close Moduli Did Not Break the Symmetry

The hope behind close moduli was that

$$r_2 = r_1 + \Delta_2, \quad r_3 = r_1 + \Delta_3$$

might make the residue information progressively more informative.

They do. But the new equations are still consequences of the same pair (p, q) satisfying the same identity $pq = n$.

The small modulus gaps permit elegant algebraic relationships such as

$$p \bmod r_2 \equiv (p \bmod r_1) - k\Delta_2 \pmod{r_2},$$

which can make coordinate transformations unusually compact. Yet the quotient k remains the representation of the unknown location of p .

In this sense, the close moduli reduce *coordinate complexity*, not necessarily *factor-location complexity*.

This distinction explains why the experiments could repeatedly produce spectacular-looking local collapses while failing to produce a fundamentally new factoring algorithm.

15 Information Content and Coordinate Changes

It is tempting to describe the outcome by saying that the “entropy of the factor” is conserved. As an intuition, this is useful, but it should not be interpreted as a formal information-theoretic theorem without a precise model.

A more defensible statement is:

The coordinate transformations investigated here did not demonstrate a reduction in the intrinsic unknown-factor search dimension. They mainly redistributed the unknown information among residues, quotients, intervals, congruence classes, defects, or solver variables.

Examples illustrate the pattern.

A factor p became

$$p = a + kr_1.$$

The unknown location then appeared in (a, k) . A different representation placed it in quotient indices (k, ℓ) . A defect representation transferred it into a divisibility condition. A CRT representation encoded it into a residue class depending on p . A symbolic formulation moved the search into the solver’s internal state.

The representation changed, but the missing factor location remained missing.

16 What the Experiments Actually Established

Several conclusions are supported by the combined research.

First, multiple modular congruences are highly effective necessary-condition filters. Once a factor candidate is known or partially specified, the complementary factor can often be reconstructed or sharply constrained.

Second, the hyperbola geometry is a genuine one-dimensional reduction of the two-factor product relation. For each fixed p interval, the possible q interval is narrow and explicitly computable.

Third, close moduli produce useful coordinate identities. These identities can turn nonlinear-looking relations into linear congruences in one quotient variable once other coordinates are fixed.

Fourth, several apparent arithmetic “collapses” can be exactly traced back to the original factorization condition. In particular, the quotient-cell defect division condition reduces to $q \mid n$.

Fifth, an enormous reduction in the number of survivors after several modular filters does not imply a fast factoring algorithm if the initial candidate domain still has to be generated.

Sixth, symbolic constraint solving can remove explicit enumeration from the source code while leaving the underlying combinatorial search intact. Solver performance must therefore be analyzed as an algorithmic search process, not merely by counting loops in the surrounding program.

17 What Is Missing?

The experiments identify a very specific missing component.

The modular equations provide relations of the form

$$q \equiv f(p) \pmod{R}.$$

The hyperbola provides

$$q = \frac{n}{p}.$$

The CRT provides a compact representation of compatible residues. The quotient and defect transformations provide alternative coordinates.

What is missing is a mechanism that maps

$$n$$

to a small region of the factor space without first traversing that space.

In other words, the missing step is a mathematically efficient *generator*.

A successful next-generation approach would need to produce something comparable to

$$n \longrightarrow \{p_1, p_2, \dots, p_m\},$$

where m is small and the set is obtained without enumerating an essentially comparable number of possible factors.

The current modular constructions instead provide

$$\{p_1, p_2, \dots, p_M\} \longrightarrow \{\text{few survivors}\}.$$

The direction of the arrow is the fundamental distinction.

18 Relation to Broader Factoring Methods

The research therefore points toward a broader methodological lesson.

Successful general-purpose factoring algorithms do not rely solely on exact algebraic re-expression of $pq = n$. They exploit additional global structure that creates useful objects independently of a guessed factor location.

For example, modern approaches such as the Number Field Sieve exploit smoothness relations and large-scale algebraic dependencies. Lattice-based approaches such as Copper-Smith's method exploit partial information about a factor or a polynomial root. The important common feature is that they introduce a new source of structure that is not merely a coordinate change of the original factor pair.

The present experiments, by contrast, remained within the closed world of

$$pq = n,$$

its modular consequences, and its geometric consequences. They consequently tended to preserve the same underlying search burden.

This does not make modular arithmetic uninteresting. It means that modular arithmetic alone is unlikely to be the missing engine unless additional structure changes the nature of the problem.

19 Future Research Direction

The most promising future investigations should therefore be selected according to whether they introduce genuinely new information rather than another equivalent coordinate system.

A useful candidate direction would be to search for a property of n whose derivation is computationally cheaper than factoring but whose consequences distinguish factor locations. Such a property would have to do more than compute another congruence implied by $pq = n$.

Another direction is to investigate whether the close-modulus structure can interact with a genuinely global method: for example, lattice reduction, polynomial root finding, smoothness relations, or another mechanism that constructs a small candidate set without enumerating the factor interval.

A third direction is algorithmic rather than algebraic: determine exactly when modular candidate generation can be implemented as a direct arithmetic progression, interval intersection, or lattice-point enumeration whose total cost is provably smaller than the factor search it replaces. The crucial requirement is that the generation cost itself, not merely the filtering cost, must become sublinear relative to the original candidate domain.

20 Conclusion

The research began with the possibility that several closely spaced prime moduli could expose hidden structure in the factorization problem. A long sequence of increasingly specialized experiments tested that idea through residues, quotient coordinates, close-modulus linearization, mixed quotient scales, hyperbola strips, residue lifting, quotient-cell defects, batch divisor isolation, CRT reconstruction, and symbolic constraint solving.

The experiments did not reveal a new factor-generation mechanism. Instead, they consistently revealed the same deeper structure: modular arithmetic is exceptionally good at expressing and enforcing consistency between candidate factors, but the tested modular equations do not independently reveal where an unknown factor is located.

The close-modulus relations therefore appear to provide *coordinate compression and relational filtering*, rather than an independent source of factor location.

The most important conceptual result is the distinction

$\text{constraint/filtering} \neq \text{candidate generation.}$

The research repeatedly achieved the first. It did not establish the second.

This boundary is itself useful. It prevents further work from repeatedly rediscovering the same structure under new variable names, and it gives a concrete criterion for evaluating future ideas: a new construction is genuinely promising only when it generates substantially fewer possible factor locations *before* those locations are enumerated, tested, or implicitly searched.

The remaining problem is therefore sharply defined. The challenge is no longer to find another way of writing

$$pq \equiv n \pmod{R}.$$

The challenge is to discover an additional mathematical engine that can turn information about n into a small, efficiently generated set of possible factor locations.

Acknowledgements of Scope

This paper records conclusions drawn from the described experimental sequence and intentionally omits numerical data analysis, benchmark tables, and performance statistics. The conclusions are therefore qualitative and structural. They should be interpreted as conclusions about the tested families of constructions rather than as a proof that no other modular formulation could ever yield a breakthrough in integer factorization.